

MONITOR Z · L03

Hablarle a JES por SSI 80

Cuaderno de estudio · versión del 2026-10-05

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L03 — Hablarle a JES por SSI 80

Objetivo: entender cómo un programa le pregunta a JES2 qué jobs están ejecutando y cuáles terminaron, cómo se recorre la respuesta y por qué hay que **devolver la memoria**. Es lo que hace ZTKMONS en cada vuelta.

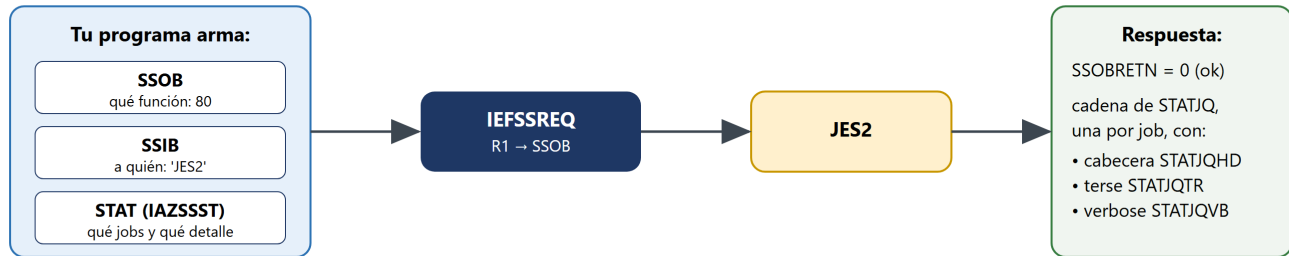
1. Qué es el SSI

El **SSI** (*Subsystem Interface*) es la ventanilla por la que un programa le pide cosas a un subsistema, en nuestro caso JES2. Cada trámite tiene un **código de función**; el **80** es *extended status*: el estado de los jobs, con mucho más detalle que el viejo "status".

Tres datos importantes:

- Se llama con la macro `IEFSSREQ`, con R1 apuntando a una palabra que contiene la dirección del SSOB.
- La macro necesita los mapeos **CVT** e **IEFJESCT**, porque busca la rutina del SSI a partir del CVT. Por eso ZTKMONS los incluye al principio (línea 73).
- **No requiere APF**. ZTKJACT lo usa desde `PPCE.ZTK.LOAD` con AC=0, en estado problema.

 Fuente: *z/OS MVS Using the Subsystem Interface* (función 80, *extended status*) · cabecera de `ZTKMONS.asm`.



Y al terminar de leerla: otro pedido con STATTYPE = STATMEM para DEVOLVER esa memoria.

🏠 Analogía: la ventanilla del banco

El **SSOB** es el formulario de trámite ("quiero consultar estados", función 80), el **SSIB** dice a qué ventanilla va (JES2) y el **STAT** es la hoja con el detalle de la consulta (qué jobs, qué fase). JES te devuelve una **carpeta con fichas encadenadas**, una por job. Cuando terminás de leerlas, **devolvés la carpeta** (STATMEM); si no, se acumulan en tu escritorio (memoria perdida).

Fuente: z/OS MVS Using the Subsystem Interface (SSI function code 80, extended status) · mapeo IAZSSST · ZTKMONS.asm (LLAMASSI)

Lámina L03-01_ventanilla_ssi

2. Armar el pedido: SSOB, SSIB y STAT

Bloque	Mapeo	Para qué
SSOB	IEFSSOBH	El "formulario": qué función se pide y dónde están los otros bloques
SSIB	IEFJSSIB	A qué subsistema va: JES2
STAT	IAZSSST	El detalle de la consulta de la función 80: qué jobs, qué fase y qué nivel de detalle

Así lo arma ZTKMONS (rutina INICIA, líneas 261–285):

```

MVC  QSSOB.SSOBID,=C'SSOB'
MVC  QSSOB.SSOBLEN,=Y(SSOBHSIZ)
MVC  QSSOB.SSOBFUNC,=Y(SSOBESTA)    función 80: extended statu
ST   R3,QSSOB.SSOBSSIB              → SSIB
ST   R4,QSSOB.SSOBINDV              → STAT (el área de la func
MVC  QSSIB.SSIBID,=C'SSIB'
MVC  QSSIB.SSIBLEN,=Y(SSIBSIZE)
MVC  QSSIB.SSIBSSNM,=CL4'JES2'      a quién
MVC  QSTAT.STATEYE,=C'STAT'
MVC  QSTAT.STATLEN,=Y(STATSIZE)
MVI  QSTAT.STATVERL,STATCVRL        versión del área
MVI  QSTAT.STATVERM,STATCVRM
MVI  QSTAT.STATSEL1,STATSJBN        filtro: nombre de job ...
MVC  QSTAT.STATJOBN,PREFIJO         ... con comodines (ZTKD*)
MVI  QSTAT.STATSEL2,STATSJOB        sólo batch
MVI  QSTAT.STATSEL3,STATSPHZ        filtro por fase (STATPHAZ)

```

Antes de armar, **todo el área se limpia con MVCL** (líneas 261–265): un campo que quedó sucio de la vuelta anterior puede cambiar el filtro sin que te des cuenta.

Campo	Valores que usamos
STATTYPE	STATTERS (resumen) · STATVRBO (detalle) · STATMEM (liberar memoria)
STATPHAZ	STAT_EXEC (253, "no terminó de ejecutar") · STAT_POSTEX (254, "terminó")

 Fuente: ZTKMONS.asm rutina INICIA · mapeos IEFSSOBH, IEFJSSIB e IAZSSST.

Bloque	Campo	Para qué
SSOB	SSOBFUNC	Función pedida: 80 (extended status)
SSOB	SSOBSSIB · SSOBINDV	Dirección del SSIB · dirección del área de la función (el STAT)
SSOB	SSOBRETN	Código de retorno de la función (además del R15 de IEFSSREQ)
SSIB	SSIBSSNM	Nombre del subsistema: JES2
STAT	STATTYPE	STATTERS (resumen) · STATVRBO (detalle) · STATMEM (liberar memoria)
STAT	STATSEL1 + STATJOBN	Filtrar por nombre de job; admite comodines (ZTKD*)
STAT	STATSEL2 = STATSJOB	Sólo jobs batch (no STC ni TSU)
STAT	STATSEL3 + STATPHAZ	Filtrar por fase: STAT_EXEC (253, "no terminó") o STAT_POSTEX (254, "terminó")
Terse	STTRPHAZ	Fase detallada del job; ejecutando = STAT_ONMAIN (14) (a confirmar en la demo)
Terse	STTRXIND · STTRMXCC	Cómo terminó (normal,abend,JCL...) · máximo CC
Verbose	STVBXTST · STVBXTET	Hora de inicio y de fin de la ejecución (centésimas desde medianoche) (a confirmar sin APF)

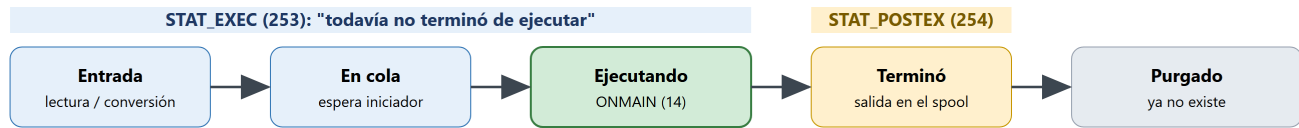
Fuente: ZTKMONS.asm (comentarios de cabecera y LLAMASSI) · mapeo IAZSSST · z/OS MVS Using the Subsystem Interface

Lámina L03-02_campos_clave

3. Las fases de un job

Un job pasa por **entrada** (lectura y conversión) → **en cola** (espera iniciador) → **ejecutando** → **terminó** (la salida queda en el spool) → **purgado**.

- STAT_EXEC** (253) abarca todo lo que **todavía no terminó**. Dentro de eso, el job que está realmente ejecutando tendría **STTRPHAZ = STAT_ONMAIN** (14). **Esto es la pregunta 2 de la demo**. Si resulta falso, ZTKMONS escribe **+DIAG FASE** en vez de **+EVT INI**.
- STAT_POSTEX** (254) es "terminó de ejecutar". De ahí salen los FIN, incluso de jobs que duraron menos que el intervalo.



Por eso, en cada vuelta, ZTKMONS pide EXEC y mira cuáles tienen fase ONMAIN (→ **INI**), y pide POSTEX para ver cuáles terminaron desde la vuelta anterior (→ **FIN**). Así ve también los jobs que duran menos que el intervalo.

✈ Analogía: el tablero del aeropuerto

Un vuelo pasa por "check-in", "embarcando", "en vuelo", "atterizó" y "fuera del tablero". JES tiene un tablero igual para los jobs. El monitor mira el tablero cada pocos segundos y anota los vuelos que **despegaron** (INI) y los que **atterizaron** (FIN).

Los valores 253, 254 y 14 salen de los comentarios de ZTKMONS sobre IAZSSST. Que el ejecutando sea siempre 14 es la **pregunta 2** de la demo.

Fuente: ZTKMONS.asm (cabecera: STAT_EXEC 253, STAT_POSTEX 254, STAT_ONMAIN 14) · mapeo IAZSSST

Lámina L03-03_fases_de_un_job

4. Leer la respuesta: una cadena de bloques

JES deja la respuesta en memoria y la cuelga del área STAT:

Paso	Campo	Qué es
1	STATJOBFB	Dirección del primer STATJQ (un bloque por job)
2	STJQNEXT	Del STATJQ al siguiente ; 0 = fin
3	STJQOHDR	Desplazamiento desde el STATJQ a su cabecera (STATJQHD)
4	STHDLEN	Largo de todo el bloque de datos del job, no sólo de la cabecera
5	Secciones	Después de la cabecera (STHDSIZE) vienen secciones, cada una con largo (2 bytes) y tipo (1 byte); la terse es STTRTERS
6	STJQVRBO	En una consulta STATVRBO: dirección de la parte verbose del job (secciones de tipo STVBVRBO)

La trampa que ya costó en ZTKJACT (comentario de BUSCATRS, línea 616):

STHDLEN es el largo de **todo** el bloque. Hay que saltar la cabecera con STHDSIZE y **recorrer las secciones por su largo y su tipo**, sin suponer que la terse viene primero.

BUSCATRS	LH	R8,QJQ.STJQOHDR	desplazamiento a la cabecera
	AR	R8,R6	R8 = cabecera
	LH	R5,QHD.STHDLEN	largo TOTAL
	LA	R8,STHDSIZE(,R8)	salto la cabecera
	LA	R7,STHDSIZE	
BTSECC	CR	R7,R5	¿me pasé del total?
	BNL	BTNO	
	LH	R0,0(,R8)	largo de esta sección
	CLI	2(R8),STTRTERS	¿es la terse?
	BER	R14	sí → R8 apunta a ella
	AR	R8,R0	no → siguiente sección
	AR	R7,R0	
	B	BTSECC	

Los campos que se leen al final:

Sección	Campo	Para qué
terse	STTRPHAZ	Fase detallada
terse	STTRXIND · STTRMXCC	Cómo terminó (normal,abend,JCL...) · máximo CC
verbose	STVBXTST · STVBXTET	Hora de inicio y de fin de la ejecución, en centésimas desde medianoche. Pregunta 4 de la demo: ¿llegan sin APF?

5. Códigos de retorno y devolver la memoria

Cada pedido deja **tres** códigos, y ZTKMONS los guarda los tres para el +ERR SSI :

Código	Dónde	Qué dice
R15 después de IEFSSREQ	RCSSI	Si el SSI aceptó el pedido (0) o no: subsistema inexistente, inactivo o que no soporta la función (constantes SSRT... de IEFSSOBH)
SSOBRETN	RCSUBSYS	Cómo le fue a la función 80
STATREAS	—	El motivo, si la función no pudo

Devolver la memoria: la respuesta ocupa memoria que JES obtuvo para vos. Después de leerla hay que hacer **otro** pedido con STATTYPE = STATMEM (rutina LIBERA, líneas 300–307). Si no, en un monitor que pregunta cada 5 s toda la noche, la memoria se acumula vuelta tras vuelta.

6. Una vuelta de ZTKMONS

#	Pedido	Qué obtiene	Qué escribe
0	Al arrancar: POSTEX sin escribir	Los jobs que ya habían terminado	+BASE YA TERMINADOS N=...
1	STATTERS , fase EXEC	Tabla NUEVA de jobs ejecutando	—
2	Comparar NUEVA con VIEJA	Jobid nuevo = arrancó	+EVT INI ...
3	STATVRBO , fase POSTEX	Terminados, con horas y CC	+EVT FIN ... de los que no estaban en REPORT
4	STATMEM	Devolver la memoria	—
5	NUEVA pasa a ser VIEJA	—	Cada 12 vueltas, +PONG

Límites conocidos de v0:

- Tablas fijas: 200 ejecutando y 2.000 reportados.
- T/TI llevan la fecha de hoy, así que un job que cruza la medianoche sale con la fecha del día siguiente.

Una vuelta de ZTKMONS, paso a paso

L03 · Hablarle a JES por SSI 80

#	Pedido	Qué obtiene	Qué escribe
0	Al arrancar: POSTEX sin escribir	Los jobs que ya habían terminado	+BASE YA TERMINADOS N=... (no se reportan)
1	STATTERS , fase EXEC, prefijo del PARM	Tabla NUEVA de jobs ejecutando	—
2	Comparar NUEVA con VIEJA	Jobid en NUEVA y no en VIEJA = arrancó	+EVT INI ...
3	STATVRBO , fase POSTEX	Jobs terminados, con horas STVBXTST/XTET y CC	+EVT FIN ... de los que no estaban en REPORT
4	STATMEM	Devolver la memoria de las respuestas	—
5	NUEVA pasa a ser VIEJA	—	Cada 12 vueltas: +PONG (latido)

Analogía: dos fotos y el juego de las diferencias

Cada vuelta saca una **foto** de quién está ejecutando (NUEVA) y la compara con la foto anterior (VIEJA): los que aparecen son INI. Aparte lleva una **libreta** (REPORT) con los FIN que ya anotó, para no repetirlos.

Límites conocidos de v0: tablas fijas (200 ejecutando, 2.000 reportados) y fecha de hoy en T/TI (un job que cruza la medianoche sale con fecha del día siguiente).

Fuente: ZTKMONS.asm (cabecera "COMO SABE QUE PASO" y "LIMITES CONOCIDOS")

1. ¿Para qué sirven por separado el R15 de `IEFSSREQ` y el `SSOBRETN`? Poné un ejemplo donde el primero dé 0 y el segundo no.
2. ¿Qué pasaría si un monitor que pregunta cada 5 s nunca hiciera el pedido `STATMEM`?
3. ¿Por qué `ZTKMONS` pide `POSTEX` una vez al arrancar **sin escribir** nada?
4. Un job dura 2 s y el intervalo es de 5 s. ¿Sale su `+EVT INI`? ¿Y su `+EVT FIN`? ¿Por qué?
5. ¿Cuál es "la trampa de `ZTKJACT`" y qué pasaría si se ignorara?

Mis respuestas

Un programa que hace **una sola** consulta STATTERS, fase EXEC, con el prefijo del PARM. Escribe una línea por job (`jobname jobid dueño clase fase`) y termina, liberando la memoria con STATMEM.

1. Copiá INICIA, LLAMASSI, LIBERA y BUSCATRS de ZTKMONS.
2. Corrélo con `PARM='*'` mientras tenés otro job tuyo ejecutando, por ejemplo tu ZTKBUCLE de P02.
3. Traé el SYSPRINT y decime si **tu** ZTKBUCLE salió con fase 14 (ONMAIN).
Esa respuesta **adelanta la pregunta 2 de la demo**.

Relacionado: [L02 - Dormir sin gastar CPU - STIMER, WAIT, POST y ECB](#) ·
[Glosario](#) · siguiente: [L04 - ZTKMONS v0 en DESA - ensamblar y la demo en modo spool](#).

 Fuente: `vivo\host\ZTKMONS.asm` (INICIA, LLAMASSI, LIBERA, ERRSSI, BUSCATRS, BUSCAVRB) · *z/OS MVS Using the Subsystem Interface* · mapeo IAZSSST.

